

# Topic 5.3

## 5.3: Conversions Between User-defined Types

When we wish to convert between two class types, the approach is similar to converting between a basic type and a class type. There are now two possibilities. Consider the statements:

```
Class Whatchamacallit
{
    ...
};

Class Thingamajig
{
    ...
};

Whatchamacallit w;
Thingamajig t;

w = t;
```

The conversion can occur in a constructor in class [Whatchamacallit](#) with a single parameter of type [Thingamajig](#):

```
Class Whatchamacallit
{
    ...
    Whatchamacallit( Thingamajig& t );
    ...
};
```

The conversion can also occur in a conversion operator in class [Thingamajig](#):

```
Class Thingamajig
{
    ...
    // New conversion operator.
    operator Whatchamacallit(void);
    ...
};
```

The following example illustrates this, converting values between rectangular coordinates and polar coordinates.

```
// MODULE:      convert1.cpp
// PROGRAMMER: Gary J. Blair
```

```

// DATE:      4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    convert1.cpp, dist4.h, dist4.cpp
// PURPOSE:
//   Represent planar coordinates
//   in rectangular and polar form.
//   Illustrate interclass conversions
//   in source class.

#include <iostream>
#include <stdlib.h>
#include "dist4.h"
#include <math.h>

using namespace std;

class Rectangular;
class Polar;

class Rectangular
{
public:
    Rectangular(void) { x=0.0; y=0.0; };
    Rectangular( double x_coord, double y_coord );
    friend ostream& operator
        <<( ostream& str, Rectangular& r );

private:
    double x;    // The x coordinate.
    double y;    // The y coordinate.
};

class Polar
{
public:
    Polar(void) { radius=0.0; angle=0.0; };
    Polar( double r, double theta );
    friend ostream& operator
        <<( ostream& str, Polar& p );

    // Conversion operator to Rectangular.
    operator Rectangular();

private:
    double radius;    // Radius from origin.
    double angle;     // Angle with conventional x-axis.
};

int main(void)
{
    const double    pi = acos(-1.0);    // Define pi.
    Polar           p1(10.0,pi/4);
    Polar           p2(20.0,pi/6);
    Rectangular     r1;

    r1 = p1;
    cout << p1 << " = " << r1 << endl;

    r1 = p2;
    cout << p2 << " = " << r1 << endl;

    return 0;
}

```

## Output from program convert1.cpp:

```
radius:10  angle:0.785398 = x:7.07107  y:7.07107
radius:20  angle:0.523599 = x:17.3205  y:10
```

```
Rectangular::Rectangular( double x_coord, double y_coord )
{
    x = x_coord;
    y = y_coord;
}
```

```
ostream& operator<<( ostream& str, Rectangular& r )
{
    str << "x:" << r.x << "  y:" << r.y;
    return str;
}
```

```
Polar::Polar( double r, double theta )
{
    radius = r;
    angle  = theta;
}
```

// Conversion operator to Rectangular.

```
Polar::operator Rectangular()
{
    double local_x = radius * cos(angle);
    double local_y = radius * sin(angle);
    return Rectangular(local_x,local_y);
}
```

```
ostream& operator<<( ostream& str, Polar& p )
{
    str << "radius:" << p.radius << "  angle:"
        << p.angle;
    return str;
}
```

```
// MODULE      convert2.cpp
// PROGRAMMER: Gary J. Blair
// DATE:       4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    convert2.cpp
// PURPOSE:
//   Represent planar coordinates in
//   rectangular and polar form.
//   Illustrate interclass conversions
//   in destination class.
```

```
#include <iostream>
#include <stdlib.h>
#include <math.h>
```

```
using namespace std;
```

```
class Rectangular;
class Polar;
```

```
class Rectangular
{
public:
```

```

Rectangular(void) { x=0.0; y=0.0; };
Rectangular( double x_coord, double y_coord );

    // Construct a Rectangular from a Polar.
    // Also serves as a convertor.

Rectangular( Polar& p );

friend ostream& operator
    << ( ostream& str, Rectangular& r );

private:
double x;        // The x coordinate.
double y;        // The y coordinate.
};

class Polar
{
public:
Polar(void) { radius=0.0; angle=0.0; };
Polar( double r, double theta );

    // Inquiry functions required by
    // Rectangular::Rectangular( Polar& p);

double Angle(void) { return angle; };
double Radius(void) { return radius; };

friend ostream& operator << ( ostream& str,
                               Polar& p );

private:
double radius;    // Radius from origin.
double angle;     // Angle with conventional
                  // x-axis.
};

int main(void)
{
    const double    pi = acos(-1.0);    // Define pi.
    Polar           p1(10.0,pi/4);
    Polar           p2(20.0,pi/6);
    Rectangular     r1;

    r1 = p1;
    cout << p1 << " = " << r1 << endl;

    r1 = p2;
    cout << p2 << " = " << r1 << endl;

    return 0;
}

```

## Output from program convert2.cpp:

```

radius:10  angle:0.785398 = x:7.07107  y:7.07107
radius:20  angle:0.523599 = x:17.3205  y:10

```

```

Rectangular::Rectangular( double x_coord, double y_coord )
{
    x = x_coord;

```

```

    y = y_coord;
}

// Construct a Rectangular from a Polar.
// Also serves as a convertor.

Rectangular::Rectangular( Polar& p )
{
    double r = p.Radius();
    double theta = p.Angle();

    x = r * cos(theta);
    y = r * sin(theta);
}

ostream& operator<<( ostream& str, Rectangular& r )
{
    str << "x:" << r.x << " y:" << r.y;
    return str;
}

Polar::Polar( double r, double theta )
{
    radius = r;
    angle = theta;
}

ostream& operator<<( ostream& str, Polar& p )
{
    str << "radius:" << p.radius << " angle:" << p.angle;
    return str;
}

```

The following example illustrates essentially all possible ways to perform conversions and where those conversions can be applied. It uses differential compilation to avoid being huge, and must be compiled and run (with a change) 3 times to get full results. The source code annotates which conversions work in each particular situation.

```

// MODULE      testconv.cpp
// PROGRAMMER: Gary J. Blair
// DATE:       4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    testconv.cpp
// PURPOSE:
//   Test conversion approaches.
//   constructor in destination class.
//   conversion function in source class.
//   overloaded operator=.
//   in the following situations:
//   assign source class object to destination class object.
//   pass sco actual to dco formal
//   use in expression
// Notes:
//   Existence of both constructor and operator
//   does not create a problem unless you attempt
//   to use one of them.

#include <iostream>
#include <stdlib.h>

```

```

#include <math.h>

#define USE_CONSTRUCTOR 1
#define USE_OPERATOR 0
#define USE_ASSIGNMENT 0

using namespace std;

class Rectangular;
class Polar;

class Rectangular
{
public:
    Rectangular(void) { x=0.0; y=0.0; };
    Rectangular( double x_coord, double y_coord );

#if (USE_CONSTRUCTOR)
    Rectangular( Polar& p );
#endif
#if (USE_ASSIGNMENT)
    Rectangular& operator=( Polar& p );
#endif

    Rectangular operator+( Rectangular& rhs );
    friend ostream& operator
        <<( ostream& str, Rectangular& r );

private:
    double x;        // The x coordinate.
    double y;        // The y coordinate.
};

class Polar
{
public:
    Polar(void) { radius=0.0; angle=0.0; };
    Polar( double r, double theta ) { radius=r; angle=theta;};
    double Angle(void) { return angle; };
    double Radius(void) { return radius; };

    // Conversion operator to Rectangular.

#if (USE_OPERATOR)
    operator Rectangular();
#endif

    friend ostream& operator<<( ostream& str, Polar& p );

private:
    double radius;    // Radius from origin.
    double angle;     // Angle with conventional
                     // x-axis.
};

void testfunc( Rectangular r );

Rectangular::Rectangular( double x_coord,
                          double y_coord )
{
    x = x_coord;
    y = y_coord;
}

// Construct a Rectangular from a Polar.
// Also serves as a converter.

```

```

#if (USE_CONSTRUCTOR)
Rectangular::Rectangular( Polar& p )
{
    double r = p.Radius();
    double theta = p.Angle();
    x = r * cos(theta);
    y = r * sin(theta);
}
#endif

#if (USE_ASSIGNMENT)
Rectangular& Rectangular::operator=( Polar& p )
{
    double r = p.Radius();
    double theta = p.Angle();
    x = r * cos(theta);
    y = r * sin(theta);
    return (*this);
}
#endif

Rectangular
Rectangular::operator+( Rectangular& rhs )
{
    double xsum = x + rhs.x;
    double ysum = y + rhs.y;
    return Rectangular(xsum,ysum);
}

ostream& operator<<( ostream& str, Rectangular& r )
{
    str << "x:" << r.x << " y:" << r.y;
    return str;
}

// Conversion operator to Rectangular.

#if (USE_OPERATOR)
Polar::operator Rectangular()
{
    double local_x = radius * cos(angle);
    double local_y = radius * sin(angle);
    return Rectangular(local_x,local_y);
}
#endif

ostream& operator<<( ostream& str, Polar& p )
{
    str << "radius:" << p.radius << " angle:"
        << p.angle;
    return str;
}

void testfunc( Rectangular r )
{
    cout << "In testfunc, r is " << r << endl;
}

int main(void)
{
    const double pi = acos(-1.0); // Define pi.
    Polar p1(10.0,pi/4);
    Polar p2(20.0,pi/6);
    Rectangular r1, r2(3,4), r3;

```

```
r1 = p1;
cout << p1 << " = " << r1 << endl;

r1 = p2;
cout << p2 << " = " << r1 << endl;

cout << "invoking with testfunc with r1 = "
    << r1 << endl;

// The following function is invoked correctly
// using either constructor or operator.
// The statement cannot be compiled if we use
// overloaded assignment operator=.

testfunc(p2);

r3 = p2;

// Following statement does not compile any of
// the 3 forms of conversion.

// r1 = r2 + p2;
// cout << r2 << " + " << r3 << " = " << r1 << endl;

return 0;
}
```

## Output from program testconv.cpp:

```
radius:10  angle:0.785398 = x:7.07107  y:7.07107
radius:20  angle:0.523599 = x:17.3205  y:10
invoking with testfunc with r1 = x:17.3205  y:10
In testfunc, r is x:17.3205  y:10
```

**Topic 5.4** (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-5-dot-4>)